Ain Shams University - Faculty of Engineering I-CHEP

CSE382 (UG2023) - Introduction to Machine Learning

Phase 1

May 2, 2026

Submitted to

Dr. Hossam El-Din Hassan

Eng. Ali Ahmed Ali Ahmed

Submitted by:

- **23P0303 : Ahmed Mohamed Fahmy Eldawy**
- **2301128 : Hady Mostafa Abdelaziz Mohamed**
- **23P0266 : John George Mikhael Khalil**
- **23P0043 : Mohamed wael ahmed ahmed**
- **23P0192 : Peter Maged shokry Noshy**

Table of Contents

1.0 Introduction 2
2.0 Preprocessing 2
3.0 Models 2
3.1 K-Nearest Neighbors 2
Mathematical Formulas 2
Results 3
3.2 Logistic Regression 3
Mathematical Formulas 3
Results 6
3.3 Naive Bayes 6
Mathematical Formulas 6
Results 6

GitHub : https://github.com/Ahmed-Fahmy00/Mnist-Classification

1.0 Introduction

This report presents the implementation and evaluation of three machine learning algorithms K Nearest Neighbors (KNN), Logistic Regression, and Naive Bayes for binary image classification on the MNIST handwritten digit dataset, as part of Phase 1 of the Project. Each model is built from scratch and evaluated using accuracy, precision, recall, F1-score, and confusion matrices.

2.0 Preprocessing

Normalization: Raw MNIST pixel values in [0,255] are normalized to [0,1]: **x = x/255**

Class Imbalance: Random undersampling is applied to the training split only
$N_{balanced} = \min(|C0|,|C1|)$

Feature Extraction: Three image representations were evaluated. Flatten converts each 28 × 28 image into a one-dimensional feature vector $x \in \mathbb{R}^{784}$. PCA (Principal Component Analysis) projects the data onto the top principal components while retaining 95% of the total variance, with the transformation fitted using only the training set. HOG (Histogram of Oriented Gradients) extracts gradient-based features using 4 × 4 pixel cells, 2 × 2 block normalization, and 9 orientation bins.

Train, Validation, Test Splits: The dataset was divided into three subsets: 60% for training, 20% for validation, and 20% for testing. The training set was used to fit the model, the validation set was used for hyperparameter tuning and model selection, and the test set was reserved for final performance evaluation on unseen data.

3.0 Models

3.1 K-Nearest Neighbors

KNN is a non-parametric, instance-based classifier with no explicit training phase the dataset is memorized, and all computation occurs at prediction time. Given a query sample, KNN computes its distance to every training point, selects the K closest neighbors, and assigns the majority class label. K is tuned via 5-fold cross-validation over $K \in \{1,3,5,7,9,11\}$

Mathematical Formulas

1. **Euclidean Distance (Vectorized):** Given query matrix X_q ($\ell \times d$) and training matrix X_{tr} ($m \times d$), squared distances are computed via a single matrix multiplication

$$D^2 = \left\|X_q\right\|^2 + \|X_{tr}\|^2 - 2X_q X_{tr}^{\top}$$

2. **Majority Vote (Classification Rule):** The predicted class is the most frequent label among the K neighbors $\hat{y} = \arg\max_c |\{x_i \in N_K(x): y_i = c\}|$
3. **Class Frequency Ratio (Probability Estimation):** Soft probabilities are the fraction of the K neighbors belonging to each class

$$\hat{P}(y = c|x) = \frac{count_c}{k}$$

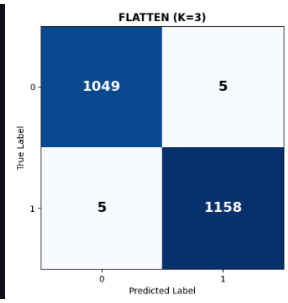
Results for class (1, 0)

```
Classification Report for FLATTEN (K=3):

   precision    recall  f1-score   support

           0    0.9953    0.9953    0.9953      1054
           1    0.9957    0.9957    0.9957      1163

    accuracy                         0.9955      2217
   macro avg    0.9955    0.9955    0.9955      2217
```



3.2 Logistic Regression

Linear, probabilistic classifier that learns weights during training. Given a sample, it computes a linear score, applies the sigmoid function to produce a probability, and assigns a class using a threshold. The model is trained using binary cross-entropy and gradient-based optimization.

Mathematical Formulas

1. **Sigmoid Activation:**

The predicted probability is computed via the sigmoid function

$$\hat{\boldsymbol{y}} = \sigma(\boldsymbol{z}) = \frac{\mathbf{1}}{\mathbf{1} + \boldsymbol{e}^{-\boldsymbol{z}}} \quad (\boldsymbol{B} \times \mathbf{1})$$

Applied element wise, $\hat{y}_i \in (0,1)$ is the predicted probability that sample i belongs to digit d

2. **Loss Function (Binary Cross-Entropy):**

For a mini-batch of size B:

$$\boldsymbol{L(w) = -\frac{1}{B}\sum_{i=1}^{B}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]}$$

The validation loss uses N_{val} = 10,000 samples and is computed after each full epoch to monitor generalization.

3. Handling Class Imbalance

To account for an imbalanced class distribution in the training data, class-specific weights are introduced into the loss function. Given N total samples, n_{pos} positive examples, and n_{neg} negative examples, the weights are:

$$w_{pos} = \frac{N}{2n_{pos}}, \quad w_{neg} = \frac{N}{2n_{neg}}$$

These factors up weight the minority class and down weight the majority class, ensuring balanced gradient signals. The weighted binary cross-entropy loss becomes:

$$L(w) = -\frac{1}{B}\sum_{i=1}^{B}\left[w_{pos} y_i \log(\hat{y}_i) + w_{neg}(1 - y_i)\log(1 - \hat{y}_i)\right]$$

4. Gradient Derivation via Chain Rule

Why $\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{B} X_b^\top (\hat{y} - y)$

We derive the gradient from first principles using the chain rule through three composed functions.

Step 1 Loss w.r.t. $\hat{y}_i$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -\frac{1}{B}\left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i}\right) = -\frac{1}{B} \cdot \frac{y_i - \hat{y}_i}{\hat{y}_i(1 - \hat{y}_i)}$$

Step 2 $\hat{y}_i$ **w.r.t.** z_i **(sigmoid derivative)**

$$\frac{\partial \hat{y}_i}{\partial z_i} = \sigma(z_i)\big(1 - \sigma(z_i)\big) = \hat{y}_i(1 - \hat{y}_i)$$

Proof. Let $\sigma = (1 + e^{-z})^{-1}$:

$$\frac{d\sigma}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma \cdot (1 - \sigma)$$

Step 3 z_i **w.r.t.** w

$$\frac{\partial z_i}{\partial w} = x_i{}' \quad (785 \times 1)$$

Chaining Steps 1–3

$$\frac{\partial \mathcal{L}}{\partial w} = \sum_{i=1}^{B} \underbrace{\frac{\partial \mathcal{L}}{\partial \hat{y}_i}}_{\text{Step 1}} \cdot \underbrace{\frac{\partial \hat{y}_i}{\partial z_i}}_{\text{Step 2}} \cdot \underbrace{\frac{\partial z_i}{\partial w}}_{\text{Step 3}}$$

Substituting:

$$\frac{\partial \mathcal{L}}{\partial w} = \sum_{i=1}^{B} \left(-\frac{1}{B} \cdot \frac{y_i - \hat{y}_i}{\hat{y}_i(1 - \hat{y}_i)} \right) \cdot \hat{y}_i(1 - \hat{y}_i) \cdot x_i{}'$$

The $\hat{y}_i(1 - \hat{y}_i)$ terms cancel exactly:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i)\ x_i{}'$$

Writing this as a matrix product:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{B}\ X_b^{\top}(\hat{y} - y) \qquad \underbrace{(785 \times B)}_{X_b^{\top}} \cdot \underbrace{(B \times 1)}_{(\hat{y} - y)} = \underbrace{(785 \times 1)}_{\nabla w}$$

This matches `de_dw = np.dot(xi_batch.T, error) / batch_size` exactly, where `error = y_hat - y`.

5. Weight Update Rule

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \cdot \frac{\partial \boldsymbol{L}}{\partial \boldsymbol{w}} \ \ (785{\times}1)$$

Matching w -= learning_rate*de_dw, where η is learning_rate.

6. Mini-Batch Training

Each epoch proceeds as follows:

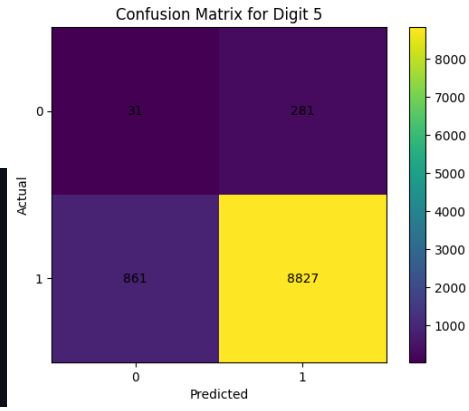
1. **Shuffle**: draw a random permutation of the 50,000 training indices indices = np.random.permutation(
2. **Batch loop**: for batch k = 0,1,…,⌊N/B⌋ − 1:

   start = k · B, end = start + B

   Extract $X_b = X_{shuffled}$ [start : end] (B×785) and y_b (B×1), then perform the forward pass and weight update described above.

Results for Class 5

```
Results for digit 5 (threshold=0.5):
  Accuracy:  96.88%
  Precision: 0.7539
  Recall:    0.9652
  F1 Score:  0.8466
```



3.3 Naive Bayes

probabilistic classifier with a fast-training phase that computes class priors and feature statistics. It assumes feature independence and models each feature with a Gaussian distribution. Given a sample, it computes class probabilities and selects the most likely label.

Mathematical Formulas

1. **Posterior Maximization:** The classifier predicts the class by maximizing the posterior probability using Bayes theorem

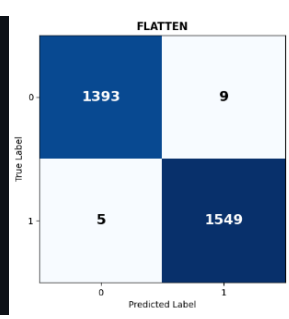
$$P(y|x) \propto P(y) \prod_i P(x_i|y)$$

2. **Log Probability Form:** prevent numerical underflow, logarithms are used

$$\hat{y} = \arg\max_c \left(\ln P(y = c) + \sum_i \ln P(x_i|y = c) \right)$$

3. **Gaussian Likelihood:** Each feature (pixel) is modeled using a Gaussian distribution

$$P(x_i|y = c) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

Results for class (1, 0)

```
Classification Report for FLATTEN:

   precision    recall  f1-score   support

           0    0.9964    0.9936    0.9950      1402
           1    0.9942    0.9968    0.9955      1554

    accuracy                         0.9953      2956
   macro avg    0.9953    0.9952    0.9953      2956
```